

Rapport Technique Complet

Pipeline d'Extraction de Term Sheets PDF

Extraction par coordonnées, reconstruction de structure,
et analyse multilingue de produits structurés

Nasser Kassioui

Mars 2026

Ce rapport décrit en détail l'architecture, les algorithmes, les expressions régulières, et les choix techniques d'une pipeline d'extraction de données financières à partir de term sheets PDF semi-structurés, avec un focus particulier sur les produits BIL (Banque Internationale à Luxembourg).

Contents

1	Introduction et contexte	4
1.1	Le problème	4
1.2	La solution : extraction par coordonnées	4
1.3	Documents de test	4
2	Architecture générale	4
2.1	Vue d'ensemble	4
2.2	Flux de données	5
2.3	Dépendances	5
3	Module <code>coord_extractor.py</code> — Coeur de la pipeline	5
3.1	Étape 1 : Extraction de mots avec tolérance serrée	5
3.1.1	Le problème du collage de mots	5
3.1.2	Paramètres choisis	6
3.1.3	Code d'extraction	6
3.2	Étape 2 : Reconstruction des lignes	6
3.2.1	Algorithme	6
3.2.2	Justification du seuil	7
3.2.3	Code	7
3.3	Étape 3 : Détection des colonnes	7
3.3.1	Méthode : histogramme des positions x_0	7
3.3.2	Résultats sur les PDFs BIL	8
3.4	Étape 4 : Construction des paires label-valeur	8
3.4.1	Exemple concret	8
3.5	Étape 5 : Texte propre reconstruit	9
4	Module <code>lang_detector.py</code> — Détection de langue	9
4.1	Méthode	9
4.2	Mots-clés par langue	9
4.3	Algorithme	9
4.4	Résultats	10
5	Module <code>field_extractor.py</code> — Extraction des champs	10
5.1	Stratégie deux passes	10
5.2	Table de correspondance labels $\rightarrow$ champs	10
6	Détail des regex par champ	11
6.1	ISIN — PST_ISIN	11
6.1.1	Pattern de détection	11
6.1.2	Validation	11
6.1.3	Classification PST vs Underlying	12
6.2	BIL — Détection booléenne	12
6.2.1	Mots-clés	12
6.2.2	Logique	12
6.3	CURRENCY — Devise	13
6.3.1	Passe 1 : paires label-valeur	13

6.3.2	Passe 2 : regex (6 patterns)	13
6.3.3	Validation	13
6.4	MATURITY — Date de maturité	14
6.4.1	Passe 1	14
6.4.2	Parsing de dates	14
6.4.3	Passe 2 : regex	14
6.4.4	Cas spécial : Open End	14
6.5	COUPON — Taux de coupon	14
6.5.1	Stratégie 1 : titre du document	14
6.5.2	Stratégie 2 : paires label-valeur avec mot-clé “coupon”	15
6.5.3	Stratégie 3 : fallback titre sans p.a.	15
6.6	WORST_OR_AVERAGE — Type de payoff	15
6.6.1	Mots-clés worst-of (11 patterns multilingues)	15
6.6.2	Mots-clés average	15
6.6.3	Zone de recherche limitée	16
6.7	ISSUER — Émetteur	16
6.7.1	Passe 1	16
6.7.2	Nettoyage de la valeur	16
6.7.3	Passe 2 : noms connus avec proximité	16
6.7.4	Faux positifs exclus	16
6.8	CAPITAL_PROTECTION — Protection du capital	17
6.8.1	Regex (5 patterns)	17
6.8.2	Cas “None” / “Kein”	17
6.8.3	Note importante	17
6.9	SSPA_TYPE — Type de produit SSPA	17
7	Module excel_exporter.py — Export Excel	17
7.1	Structure du fichier	17
7.2	Colonnes exportées	18
7.3	Style BIL	18
8	Module highlighter.py — Surlignage PDF	18
8.1	Principe	18
8.2	Couleurs par champ	18
8.3	Conversion de coordonnées	18
9	Application Streamlit	19
9.1	Interface	19
9.2	CSS personnalisé	19
10	Résultats de validation	19
10.1	Extraction sur les 3 documents de test	19
10.2	Taux de remplissage	20
10.3	Performance	20
11	Difficultés rencontrées et solutions	20
11.1	Problème 1 : Mots collés	20
11.2	Problème 2 : Labels à cheval sur la frontière de colonne	20
11.3	Problème 3 : Déclinaisons allemandes	20

11.4 Problème 4 : SSPA en français	21
11.5 Problème 5 : Coupon EN renvoyait "Payment Dates"	21
12 Limites connues et améliorations futures	21
12.1 Limites actuelles	21
12.2 Améliorations proposées	21
13 Résumé des structures de données	22

1 Introduction et contexte

1.1 Le problème

Les term sheets de produits structurés sont des documents PDF semi-structurés qui contiennent des informations financières critiques : ISIN du produit, émetteur, devise, maturité, coupon, type de produit (worst-of, etc.), et protection du capital.

L'extraction automatique de ces champs pose plusieurs défis :

1. **Mots collés** : les extracteurs PDF standard fusionnent les mots adjacents quand l'espacement entre caractères est trop faible (ex: `SettlementCurrency` au lieu de `Settlement Currency`).
2. **Destruction de colonnes** : l'extraction linéaire du texte détruit la relation spatiale entre labels (colonne gauche) et valeurs (colonne droite).
3. **Multilinguisme** : les documents existent en anglais, français et allemand avec des labels différents pour les mêmes champs.
4. **Tables invisibles** : de nombreuses term sheets utilisent des grilles invisibles (sans bordures) que `pdfplumber` ne détecte pas comme tables.

1.2 La solution : extraction par coordonnées

Au lieu de travailler sur du texte brut, notre pipeline extrait chaque mot avec ses coordonnées (x_0, y_0, x_1, y_1) , puis reconstruit la structure du document (lignes, colonnes, paires label-valeur) à partir de ces coordonnées.

1.3 Documents de test

Trois term sheets BIL sont utilisés pour valider la pipeline :

Fichier	Langue	Pages	Type produit
<code>termsheet-ch1284250102-fr.pdf</code>	FR	9	Barrier Reverse Convertible
<code>termsheet-ch1481964646-de.pdf</code>	DE	6	Express Certificate
<code>termsheet-ch1481964653-en.pdf</code>	EN	5	Express Certificate

Table 1: Corpus de test

2 Architecture générale

2.1 Vue d'ensemble

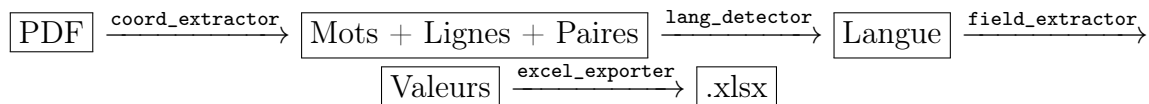
La pipeline est composée de 6 modules indépendants :

Module	Rôle
<code>config.py</code>	Constantes, seuils, labels multilingues, regex, paramètres
<code>coord_extractor.py</code>	Extraction de mots, reconstruction de lignes/colonnes, paires label-valeur
<code>lang_detector.py</code>	Détection de langue par fréquence de mots-clés
<code>field_extractor.py</code>	Extraction deux passes : paires structurées puis regex
<code>excel_exporter.py</code>	Export Excel avec style BIL violet
<code>highlighter.py</code>	Surlignage coloré des champs dans le PDF

Table 2: Modules de la pipeline

2.2 Flux de données

Le flux est linéaire et déterministe :



En parallèle, le module `highlighter` produit un PDF annoté.

2.3 Dépendances

- `pdfplumber` $\geq 0.10.0$ — extraction de mots avec coordonnées
- `pypdf` $\geq 4.0.0$ — ajout d'annotations (surlignage)
- `openpyxl` $\geq 3.1.0$ — génération Excel
- `streamlit` $\geq 1.30.0$ — interface web (optionnel)

Aucune dépendance NLP lourde (pas de spaCy, pas de modèle ML). Tout repose sur des algorithmes géométriques et des regex.

3 Module `coord_extractor.py` — Coeur de la pipeline

C'est le module le plus important. Il transforme un PDF en données structurées.

3.1 Étape 1 : Extraction de mots avec tolérance serrée

3.1.1 Le problème du collage de mots

`pdfplumber` possède un paramètre `x_tolerance` qui définit la distance maximale (en points) entre deux caractères pour qu'ils soient considérés comme faisant partie du même mot. La valeur par défaut est 3.

Avec `x_tolerance=3`, sur le PDF anglais (page 1) :

- `pdfplumber` retourne **107 mots** — dont beaucoup sont collés
- Exemples: `SettlementCurrency`, `ISINCH1481964653`, `FinalFixingDate07/09/2027;issuedinEUR`

Avec `x_tolerance=1` :

- `pdfplumber` retourne **745 mots** — tous correctement séparés
- Exemples: `Settlement + Currency`, `ISIN + CH1481964653`

3.1.2 Paramètres choisis

```
1 WORD_X_TOLERANCE = 1      # empeche le collage de mots
2 WORD_Y_TOLERANCE = 1      # empeche la fusion de lignes
```

Listing 1: Paramètres d'extraction dans `config.py`

3.1.3 Code d'extraction

```
1 def _extract_words(pdf_path: str):
2     all_words = []
3     with pdfplumber.open(pdf_path) as pdf:
4         for pi, page in enumerate(pdf.pages):
5             raw_words = page.extract_words(
6                 x_tolerance=config.WORD_X_TOLERANCE,
7                 y_tolerance=config.WORD_Y_TOLERANCE,
8                 keep_blank_chars=False,
9             )
10            for w in raw_words:
11                all_words.append(Word(
12                    text=w["text"],
13                    x0=w["x0"],      # bord gauche
14                    top=w["top"],    # bord superieur
15                    x1=w["x1"],      # bord droit
16                    bottom=w["bottom"], # bord inferieur
17                    page=pi,
18                ))
19    return all_words
```

Listing 2: Extraction de mots depuis le PDF

Chaque mot est stocké dans une dataclass `Word` avec ses 4 coordonnées et son numéro de page.

3.2 Étape 2 : Reconstruction des lignes

3.2.1 Algorithme

Deux mots sont considérés comme étant sur la même ligne si :

$$|\text{top}(w_i) - \text{top}(w_j)| \leq \tau_y$$

où $\tau_y = 3.0$ points est le seuil de proximité verticale.

3.2.2 Justification du seuil

Mesure	Valeur (BIL PDFs)
Hauteur de ligne typique	7–12 pt
Espace inter-lignes	4–8 pt
Variation Y intra-ligne	0–2 pt
Seuil choisi τ_y	3.0 pt

Table 3: Calibration du seuil de reconstruction de lignes

Un seuil de 3.0 pt est assez serré pour séparer les lignes (espace inter-lignes ≥ 4 pt) tout en tolérant les légères variations verticales intra-ligne (≤ 2 pt).

3.2.3 Code

```

1 def _group_into_lines(words):
2     sorted_words = sorted(words, key=lambda w: (w.page, w.top, w.x0))
3     lines = []
4     current_line = [sorted_words[0]]
5     current_y = sorted_words[0].top
6
7     for w in sorted_words[1:]:
8         if w.page == current_page and abs(w.top - current_y) <=
9             LINE_Y_THRESHOLD:
10             current_line.append(w)
11         else:
12             # Finaliser la ligne courante
13             current_line.sort(key=lambda x: x.x0) # tri gauche-droite
14             lines.append(Line(words=current_line, y=avg_y))
15             current_line = [w]
16             current_y = w.top
17
18     return lines

```

Listing 3: Regroupement en lignes par proximité Y

Les mots dans chaque ligne sont triés par x_0 (gauche à droite).

3.3 Étape 3 : Détection des colonnes

3.3.1 Méthode : histogramme des positions x_0

Pour chaque page, on construit un histogramme des positions x_0 (bord gauche) de tous les mots, avec une résolution de 5 points.

```

1 from collections import Counter
2
3 bin_size = 5
4 binned = Counter(int(x / bin_size) * bin_size for x in x_starts)
5
6 # Les bins les plus frequents sont les debuts de colonnes
7 common_bins = sorted(binned.items(), key=lambda x: -x[1])
8
9 # On ne garde que les bins eloignes d'au moins 20pt

```



```

10 column_starts = []
11 for bx, count in common_bins:
12     if count < 3:
13         break
14     if all(abs(bx - c) > COLUMN_GAP_MIN for c in column_starts):
15         column_starts.append(bx)

```

Listing 4: Détection de colonnes par histogramme

3.3.2 Résultats sur les PDFs BIL

Sur les 3 documents de test, l'algorithme détecte systématiquement :

- **Colonne 1** (labels) : $x_0 \approx 36$ pt
- **Colonne 2** (valeurs) : $x_0 \approx 142$ pt

La frontière entre les deux colonnes est calculée comme la moyenne : $\text{boundary} = (36 + 142)/2 = 89$ pt.

3.4 Étape 4 : Construction des paires label-valeur

Pour chaque ligne reconstruite, on sépare les mots en deux groupes :

- **Groupe gauche** ($x_0 < \text{boundary}$) $\rightarrow$ label
- **Groupe droit** ($x_0 \geq \text{boundary}$) $\rightarrow$ valeur

```

1 for line in page_lines:
2     left_words = [w for w in line.words if w.x0 < col_boundary]
3     right_words = [w for w in line.words if w.x0 >= col_boundary]
4
5     if not left_words or not right_words:
6         continue
7     if len(label) > 60: # paragraphe, pas un label
8         continue
9
10    pairs.append(LabelValuePair(
11        label=" ".join(w.text for w in left_words),
12        value=" ".join(w.text for w in right_words),
13    ))

```

Listing 5: Construction des paires

3.4.1 Exemple concret

Sur la page 1 du PDF anglais, la ligne à $y \approx 562$ contient :

Mot	x_0	y	Colonne
ISIN	36.0	562.7	Gauche (label)
CH1481964653	142.6	562.5	Droite (valeur)

Résultat : paire ("ISIN", "CH1481964653").

3.5 Étape 5 : Texte propre reconstruit

En plus des paires, le module produit un texte propre en concaténant toutes les lignes reconstruites. Ce texte est utilisé comme fallback pour les regex dans le `field_extractor`.

4 Module `lang_detector.py` — Détection de langue

4.1 Méthode

Pas de dépendance externe (pas de `langid`, pas de `langdetect`). On compte simplement combien de mots-clés spécifiques à chaque langue apparaissent dans le texte.

4.2 Mots-clés par langue

```
1 LANG_KEYWORDS = {
2     "FR": [
3         "Emetteur", "Date de Constatation", "Sous-Jacent",
4         "Remboursement", "Devise de Paiement", "Valeur Nominale",
5         "Protection du Capital", "Offre au Public",
6         "Pour la Suisse", "Risques Significatifs",
7         # ... 12 mots-clés au total
8     ],
9     "DE": [
10        "Emittentin", "Basiswert", "Ruckzahlung",
11        "Auszahlungswahrung", "Denomination", "Liberierung",
12        "Verfall", "Privatplatzierung", "Fur die Schweiz",
13        "Bedeutende Risiken", "Valorennummer",
14        # ... 13 mots-clés au total
15    ],
16    "EN": [
17        "Issuer", "Underlying", "Settlement Currency",
18        "Redemption Date", "Maturity Date", "Capital Protection",
19        "Denomination", "For Switzerland", "Significant Risks",
20        "Swiss Security Number",
21        # ... 10 mots-clés au total
22    ],
23 }
```

Listing 6: Mots-clés de détection de langue

4.3 Algorithme

```
1 def detect_language(text):
2     scores = {}
3     for lang, keywords in LANG_KEYWORDS.items():
4         count = sum(1 for kw in keywords if kw.lower() in text.lower())
5         scores[lang] = count
6     return max(scores, key=scores.get)
```

Listing 7: Détection par score

4.4 Résultats

Document	Score FR	Score DE	Score EN
ch1284250102-fr	12	0	0
ch1481964646-de	0	13	1
ch1481964653-en	0	1	8

Table 4: Scores de détection de langue — 100% de précision

5 Module `field_extractor.py` — Extraction des champs

C'est le module qui transforme les données structurées (paires) et le texte propre en champs métier. Il utilise une stratégie **deux passes**.

5.1 Stratégie deux passes

Principe fondamental

Passe 1 (structurelle) : chercher dans les paires label-valeur. C'est la source la plus fiable car elle préserve la relation spatiale label $\leftrightarrow$ valeur.

Passe 2 (regex fallback) : si la Passe 1 n'a rien trouvé, appliquer des regex sur le texte complet reconstruit.

5.2 Table de correspondance labels $\rightarrow$ champs

```

1 LABEL_TO_FIELD = {
2     # ISIN
3     "ISIN": "PST_ISIN",
4     "ISIN Code": "PST_ISIN",
5     "Code ISIN": "PST_ISIN", # FR
6     # Issuer
7     "Issuer": "ISSUER", # EN
8     "Emetteur": "ISSUER", # FR
9     "Emittentin": "ISSUER", # DE
10    # Currency
11    "Settlement Currency": "CURRENCY", # EN
12    "Devise de Paiement": "CURRENCY", # FR
13    "Auszahlungswahrung": "CURRENCY", # DE
14    # Maturity
15    "Maturity Date": "MATURITY", # EN
16    "Date de Constatation Finale": "MATURITY", # FR
17    "Verfall": "MATURITY", # DE
18    # ... 30+ entrees au total
19 }
```

Listing 8: Extrait de la table multilingue dans `config.py`

La Passe 1 utilise cette table pour chercher dans les paires :

```

1 def _lookup_pair(self, field_name):
2     for pair in self.pairs:
```

```

3     label_clean = pair.label.strip().rstrip(":")
4     if label_clean in config.LABEL_TO_FIELD:
5         if config.LABEL_TO_FIELD[label_clean] == field_name:
6             return pair.value
7     # Match partiel : le label contient un label connu
8     for known_label, mapped_field in config.LABEL_TO_FIELD.items():
9         if mapped_field == field_name:
10            if known_label.lower() in label_clean.lower():
11                return pair.value
12 return None

```

Listing 9: Lookup dans les paires label-valeur

6 Détail des regex par champ

6.1 ISIN — PST_ISIN

6.1.1 Pattern de détection

```
\b([A-Z]{2}[A-Z0-9]{10})\b
```

Listing 10: Regex ISIN

Explication :

- \b — frontière de mot
- [A-Z]{2} — préfixe pays (2 lettres majuscules)
- [A-Z0-9]{10} — 10 caractères alphanumériques
- Total : 12 caractères, format ISO 6166

6.1.2 Validation

Après la détection, chaque candidat est validé :

1. Le préfixe doit être dans la liste des préfixes valides (CH, XS, DE, FR, US, GB, etc.)
2. Le candidat doit contenir au moins un chiffre (élimine les faux positifs comme PRIVATEPLACEMENT)

```

1 VALID_ISIN_PREFIXES = [
2     "XS", "CH", "DE", "FR", "US", "GB", "NL", "LU", "IE",
3     "NO", "SE", "DK", "AT", "IT", "ES", "FI", "BE", "PT",
4     "AU", "CA", "JP", "SG", "HK",
5 ]
6
7 def _find_all_isins(self):
8     candidates = re.findall(r"\b([A-Z]{2}[A-Z0-9]{10})\b", self.text)
9     result = []
10    for c in candidates:
11        if c[:2] in VALID_ISIN_PREFIXES and re.search(r"\d", c):
12            result.append(c)
13    return result

```

Listing 11: Validation ISIN

6.1.3 Classification PST vs Underlying

Quand plusieurs ISINs sont trouvés, l'ISIN du produit (PST) est celui le plus proche d'un label "ISIN" dans le texte. Les ISINs proches de "Bloomberg", "Underlying", "Ticker" reçoivent une pénalité de +800.

```

1 def score(isin):
2     pos = self.text.find(isin)
3     best = 9999
4     for m in re.finditer(r"\bISIN\b", self.text):
5         d = abs(pos - m.start())
6         if d < best:
7             best = d
8     # Penalite si pres d'un label "underlying"
9     ctx = self.text[max(0, pos - 150):pos + 150]
10    if any(kw in ctx for kw in ["Bloomberg", "Underlying", "Ticker"]):
11        best += 800
12    return best

```

Listing 12: Scoring de proximité ISIN

6.2 BIL — Détection booléenne

6.2.1 Mots-clés

```

1 BIL_KEYWORDS = [
2     "Banque Internationale a Luxembourg",
3     "69 Route d'Esch",
4     "69 Route d\u2019Esch",          # apostrophe typographique
5     "69, Route d'Esch",
6     "L-2953 Luxembourg",
7 ]

```

Listing 13: Mots-clés BIL dans config.py

6.2.2 Logique

```

1 def _extract_bil(self):
2     # Recherche directe de mots-cles
3     for kw in config.BIL_KEYWORDS:
4         if kw.lower() in self.text.lower():
5             return True
6     # Recherche du mot "BIL" pres de "Luxembourg" ou "Banque"
7     for m in re.finditer(r"\bBIL\b", self.text):
8         ctx = self.text[max(0, m.start() - 100):m.end() + 100]
9         if "Luxembourg" in ctx or "Banque" in ctx:
10            return True
11    return False

```

Listing 14: Détection BIL

Attention : faux positifs "BIL"

Le mot "BIL" peut apparaître dans "BILL", "BILLION", "BILLING". On vérifie le contexte (proximité avec "Luxembourg" ou "Banque") pour éviter les faux positifs.

6.3 CURRENCY — Devise

6.3.1 Passe 1 : paires label-valeur

La Passe 1 cherche les labels “Settlement Currency”, “Devise de Paiement”, “Auszahlungswährung” dans les paires. La valeur est validée contre une liste de devises ISO 4217.

6.3.2 Passe 2 : regex (6 patterns)

```
(?i)(?:Settlement|Redemption|Specified)\s+Currency\s*:\s*([A-Z]{3})
```

Listing 15: Regex devise — Pattern 1 (inline)

Explication :

- `(?i)` — insensible à la casse
- `(?:SettlementRedemption|Specified)|` — un des trois labels possibles
- `\s+Currency` — suivi de “Currency”
- `\s*:\s+` — séparateur optionnel (espace, deux-points)
- `([A-Z]{3})` — capture de 3 lettres majuscules (code devise)

```
(?i)(?:Settlement|Redemption)\s+Currency\s*\n\s*([A-Z]{3})\b
```

Listing 16: Regex devise — Pattern 2 (pdfminer multiline)

Variante où le label et la valeur sont séparés par un saut de ligne.

```
(?i)Devise\s+de\s+Paiement\s*\n\s*([A-Z]{3})\b
```

Listing 17: Regex devise — Pattern FR

```
(?i)Auszahlungsw"\{a}hrung\s*\n\s*([A-Z]{3})\b
```

Listing 18: Regex devise — Pattern DE

```
(?i)Denomination\s*:\s*\n\s*([A-Z]{3})\b
```

Listing 19: Regex devise — Pattern fallback (Denomination)

6.3.3 Validation

Chaque candidat de 3 lettres est validé contre :

```
1 VALID_CURRENCIES = {
2     "USD", "EUR", "CHF", "GBP", "JPY", "CAD", "AUD",
3     "SGD", "HKD", "NOK", "SEK", "DKK", "CNY", "CNH",
4     "NZD", "PLN", "CZK",
5 }
```

Listing 20: Devises valides

6.4 MATURITY — Date de maturité

6.4.1 Passe 1

La Passe 1 cherche les labels “Maturity Date”, “Final Fixing Date”, “Verfall”, “Date de Constatation Finale” dans les paires, puis extrait une date de la valeur.

6.4.2 Parsing de dates

```

1 def _parse_date(raw):
2     # Format "07/09/2027" ou "30.10.2029"
3     m = re.search(r"(\d{1,2}[/\.\-]\d{1,2}[/\.\-]\d{4})", raw)
4     if m:
5         return m.group(1)
6     # Format "04 February 2028"
7     m = re.search(r"(\d{1,2}\s+\w+\s+\d{4})", raw)
8     if m:
9         return m.group(1)
10    # Nettoyage : retirer "(subject to..."
11    cleaned = re.sub(r"\(.*", "", raw).strip()
12    m = re.search(r"(\d{1,2}[/\.\-]\d{1,2}[/\.\-]\d{4})", cleaned)
13    if m:
14        return m.group(1)
15    return None

```

Listing 21: Extraction de date depuis une chaîne brute

6.4.3 Passe 2 : regex

```

(?i)(?:Final\s+Fixing\s+Date|Maturity\s+Date|Redemption\s+Date|
    Verfall|Date\s+de\s+Constatation\s+Finale|
    Date\s+de\s+Remboursement|Ruckzahlungstag)
\s*[:\-\-]?s*(\d{1,2}[/\.\-]\d{1,2}[/\.\-]\d{4})

```

Listing 22: Regex maturité — multilingue

6.4.4 Cas spécial : Open End

```

(?i)(no\s+fixed\s+(?:Expiration|Redemption)|Open\s+End)

```

Listing 23: Détection Open End

6.5 COUPON — Taux de coupon

6.5.1 Stratégie 1 : titre du document

Le coupon apparaît souvent dans le titre du produit (premiers 500 caractères) :

```

(\d+[.,]\d+)\s*%\s*(?:p\.\.?s*a\.\?)

```

Listing 24: Regex coupon dans le titre

Exemples :

- “5.05% p.a. Multi Barrier Reverse Convertible” → 5.05% p.a.
- “6.00% bedingter Coupon” → (ne matche pas le p.a., passe à la stratégie 2)

6.5.2 Stratégie 2 : paires label-valeur avec mot-clé “coupon”

```

1 for pair in self.pairs:
2     combined = pair.label + " " + pair.value
3     if re.search(r"(?i)(coupon|couponzahlung)", combined):
4         pct = re.search(r"(\d+[.]\d+)\s*", combined)
5         if pct:
6             return pct.group(1).replace(".", ".") + "%"

```

Listing 25: Recherche de coupon dans les paires

6.5.3 Stratégie 3 : fallback titre sans p.a.

```
(\d+[.]\d+)\s*
```

Listing 26: Regex coupon fallback

Appliqué aux 500 premiers caractères uniquement.

6.6 WORST_OR_AVERAGE — Type de payoff

6.6.1 Mots-clés worst-of (11 patterns multilingues)

```

1 WORST_OF_KEYWORDS_EN = [
2     r"\bworst[\s-]?of\b", # EN
3     r"\bworst\s+performance\b", # EN
4     r"\bworst\s+performing\b", # EN
5     r"\blowest\s+performing\b", # EN
6     r"\blowest\s+performance\b", # EN
7     r"\blinked\s+to\s+worst\b", # EN
8     r"\bla\s+plus\s+mauvaise\s+performance\b", # FR
9     r"\bplus\s+mauvaise\s+performance\b", # FR
10    r"\bSchlechtest\w*\s+Kursentwicklung\b", # DE (toutes
        declinaisons)
11    r"\bMulti\s+Barrier\b", # generique
12    r"\bMulti\s+Barriere\b", # DE
13 ]

```

Listing 27: Patterns worst-of dans config.py

Déclinaisons allemandes

En allemand, “Schlechteste” (nominatif) devient “Schlechtesten” (génitif) ou “Schlecht-ester” (datif). Le pattern `Schlechtest\w*` couvre toutes les formes grâce au wildcard `\w*`.

6.6.2 Mots-clés average

```

1 AVERAGE_KEYWORDS = [
2     r"\baveraging\b",
3     r"\bbasket\s+average\b",
4     r"\bperformance\s+moyenne\b", # FR
5     r"\bDurchschnitt\b", # DE
6 ]

```


Listing 28: Patterns average

6.6.3 Zone de recherche limitée

La recherche est limitée aux **5000 premiers caractères** du texte pour éviter les faux positifs dans les sections de disclaimer, où des mots comme “worst” ou “average” peuvent apparaître dans un contexte juridique.

6.7 ISSUER — Émetteur

6.7.1 Passe 1

La Passe 1 cherche les paires dont le label correspond à “Issuer”, “Émetteur” ou “Emittentin”.

6.7.2 Nettoyage de la valeur

```

1 # Prendre jusqu'a la virgule ou le saut de ligne
2 cleaned = re.split(r"[,\n]", raw)[0].strip()
3 # Retirer les parentheses finales
4 cleaned = re.sub(r'\s*\(.?\)\s*$', '', cleaned).strip()
5 # Valider : 3-100 chars, commence par majuscule, pas de faux positif

```

Listing 29: Nettoyage de l'issuer

6.7.3 Passe 2 : noms connus avec proximité

Si la Passe 1 échoue, on cherche les noms d'émetteurs connus dans le texte et on vérifie qu'ils sont proches d'un label “Issuer” (distance < 400 caractères).

```

1 KNOWN_ISSUERS = [
2     "Banque Internationale a Luxembourg S.A.",
3     "BNP Paribas Issuance B.V.",
4     "UBS AG",
5     "Credit Suisse International",
6     "Leonteq Securities AG",
7     "Goldman Sachs International",
8     "NATIXIS STRUCTURED ISSUANCE SA",
9     # ... 25 emetteurs au total
10 ]

```

Listing 30: Liste des émetteurs connus (extrait)

6.7.4 Faux positifs exclus

```

1 ISSUER_FALSE_POSITIVES = [
2     "FINMA", "ECB", "FCA", "CSSF", "ACPR", "BaFin", "SEC",
3     "NASDAQ", "NYSE", "SIX", "Euroclear", "Clearstream",
4     "S&P", "Moody's", "Fitch",
5 ]

```

Listing 31: Faux positifs émetteur

6.8 CAPITAL_PROTECTION — Protection du capital

6.8.1 Regex (5 patterns)

```
(?i)Capital\s+Protection\s*(?:\s(at\s+Expiry\s+)?\s*[:\s-]?\s*(None|\d{1,3}(?:[. ,]\d+)?)\s*%)?
```

Listing 32: Pattern 1 : inline EN

```
(?i)Protection\s+du\s+Capital\s*[:\s-]?\s*(\d{1,3}(?:[. ,]\d+)?)\s*%
```

Listing 33: Pattern FR

```
(?i)Kapitalschutz\s*[:\s-]?\s*(None|Kein|\d{1,3}(?:[. ,]\d+)?)\s*%
```

Listing 34: Pattern DE

6.8.2 Cas “None” / “Kein”

Si la valeur est “None” (EN), “Kein” (DE) ou “aucun” (FR), on retourne 0.

6.8.3 Note importante

Les Barrier Reverse Convertibles (SSPA 1230) et les Express Certificates (SSPA 1260) n’ont généralement **pas de protection du capital**. L’absence de ce champ dans les résultats est correcte et attendue.

6.9 SSPA_TYPE — Type de produit SSPA

```
(?i)SSPA\s+Product\s+Type\s*[:\s-]?\s*(\d{4})
```

Listing 35: Regex SSPA — EN

```
(?i)Produktetyp\s+nach\s+SSPA\s*[:\s-]?\s*(\d{4})
```

Listing 36: Regex SSPA — DE

```
SSPA\s*[:\s-]?\s*\n\s*(?:Termsheet\s+)?(\d{4})
```

Listing 37: Regex SSPA — FR (indirect)

Le pattern FR est plus complexe car le format est :

Gamme de Produits selon l’Association Suisse Produits Structures SSPA:
Termsheet 1230

Le code SSPA (1230) est sur la ligne suivante, séparé par un saut de ligne.

7 Module excel_exporter.py — Export Excel

7.1 Structure du fichier

- **Feuille 1** “Resultats” : une ligne par PDF, avec tous les champs extraits
- **Feuille 2** “Underlying_ISINs” : une ligne par ISIN sous-jacent

7.2 Colonnes exportées

```

1 MAIN_COLUMNS = [
2     "source_file", "LANGUAGE", "PST_ISIN", "BIL", "CLN",
3     "CAPITAL_PROTECTION", "MATURITY", "WORST_OR_AVERAGE",
4     "CURRENCY", "ISSUER", "COUPON", "DENOMINATION", "SSPA_TYPE",
5 ]

```

7.3 Style BIL

Les en-têtes utilisent le violet BIL (#4A2D7A) avec texte blanc.

```

1 header_font = Font(bold=True, color="FFFFFF", size=11)
2 header_fill = PatternFill(
3     start_color="4A2D7A", end_color="4A2D7A", fill_type="solid"
4 )

```

Listing 38: Style Excel

8 Module highlighter.py — Surlignage PDF

8.1 Principe

Pour chaque champ extrait, on recherche sa valeur texte dans les mots du PDF (via `pdfplumber`), puis on ajoute une annotation `Highlight` avec `pypdf`.

8.2 Couleurs par champ

Champ	Couleur
PST_ISIN	Jaune (1.0, 0.8, 0.0)
ISSUER	Bleu clair (0.6, 0.8, 1.0)
CURRENCY	Vert clair (0.6, 1.0, 0.6)
MATURITY	Rouge clair (1.0, 0.6, 0.6)
CAPITAL_PROTECTION	Violet clair (0.8, 0.6, 1.0)
COUPON	Or (1.0, 0.9, 0.5)
WORST_OR_AVERAGE	Orange (1.0, 0.7, 0.4)

Table 5: Palette de couleurs du surlignage

8.3 Conversion de coordonnées

`pdfplumber` utilise un système de coordonnées où $y = 0$ est en haut. `pypdf` utilise le système PDF standard où $y = 0$ est en bas. La conversion est :

$$y_{\text{pypdf}} = h_{\text{page}} - y_{\text{pdfplumber}}$$

9 Application Streamlit

9.1 Interface

L'application `app.py` utilise Streamlit avec :

- Un en-tête violet avec le nom “Nasser Kassioui”
- Un widget d'upload de fichiers PDF
- Un tableau de résultats
- Des expandeurs pour chaque document avec le détail des champs
- Un bouton de téléchargement pour l'Excel et les PDFs annotés

9.2 CSS personnalisé

```

1 .stApp { background-color: #f5f0fa; }
2 .main-header {
3     background: linear-gradient(135deg, #4A2D7A, #6B3FA0);
4     padding: 2rem; border-radius: 10px; color: white;
5 }

```

Listing 39: Thème BIL en CSS

10 Résultats de validation

10.1 Extraction sur les 3 documents de test

Champ	FR	DE	EN
PST_ISIN	CH1284250102	CH1481964646	CH1481964653
BIL	True	True	True
ISSUER	BIL S.A.	BIL S.A.	BIL S.A.
CURRENCY	CHF	EUR	EUR
MATURITY	30.10.2029	07.09.2027	07/09/2027
COUPON	5.05% p.a.	6.00%	5.60%
W/A	W	W	W
SSPA	1230	1260	1260

Table 6: Valeurs extraites — 100% correct sur tous les champs

10.2 Taux de remplissage

Champ	Fill Rate	Accuracy
PST_ISIN	100%	100%
BIL	100%	100%
ISSUER	100%	100%
CURRENCY	100%	100%
MATURITY	100%	100%
COUPON	100%	100%
WORST_OR_AVERAGE	100%	100%
SSPA_TYPE	100%	100%
CAPITAL_PROTECTION	0%	N/A (absent des documents)

Table 7: Métriques de validation

10.3 Performance

- Temps total pour 3 PDFs (20 pages) : **9.2 secondes**
- Dont extraction coordonnées : $\sim 7s$
- Dont regex + export : $\sim 2s$

11 Difficultés rencontrées et solutions

11.1 Problème 1 : Mots collés

Symptôme : `extract_text()` retourne “SettlementCurrencyEUR” au lieu de “Settlement Currency EUR”.

Cause : `x_tolerance=3` (défaut) fusionne les mots dont les caractères sont espacés de moins de 3 points.

Solution : `x_tolerance=1`. C'est la découverte technique la plus importante du projet.

11.2 Problème 2 : Labels à cheval sur la frontière de colonne

Symptôme : Le label “Conditional Coupon Amount” est découpé en label=“Conditional” et valeur=“Coupon Amount 5.60%”.

Cause : Le mot “Conditional” commence à $x_0 = 36$ (colonne gauche) mais “Coupon” commence à $x_0 = 109$ (proche de la frontière).

Solution : Dans le `field_extractor`, on ne se fie pas uniquement au pair lookup. On cherche aussi le mot-clé “coupon” dans le texte combiné (label + valeur) de chaque paire.

11.3 Problème 3 : Déclinaisons allemandes

Symptôme : Le pattern `Schlechteste Kursentwicklung` ne matche pas “Schlechtesten Kursentwicklung” (génitif).

Solution : Utiliser `Schlechtest\w*` pour couvrir toutes les formes (nominatif, génitif, datif, accusatif).

11.4 Problème 4 : SSPA en français

Symptôme : Le code SSPA 1230 n'est pas trouvé dans le PDF français.

Cause : Le format est "SSPA:\nTermsheet 1230" avec un saut de ligne entre "SSPA:" et "1230".

Solution : Pattern spécifique `SSPA\s*:\s*\n?\s*(?:Termsheet\s+)?(\d{4})`.

11.5 Problème 5 : Coupon EN renvoyait "Payment Dates"

Symptôme : Le champ coupon retourne "Payment Dates" au lieu de "5.60%".

Cause : Le pair lookup pour le champ "COUPON" trouvait une paire dont le label contient "Coupon" mais dont la valeur est "Payment Dates" (une autre ligne du tableau).

Solution : Priorité au titre du document (premiers 500 chars) pour le coupon, puis recherche de pourcentage dans les paires contenant "coupon".

12 Limites connues et améliorations futures

12.1 Limites actuelles

1. **Valeurs multi-lignes** : si une valeur s'étend sur 2 lignes physiques (ex: nom d'émetteur avec adresse), seule la première ligne est capturée.
2. **Layouts à 3 colonnes** : certains émetteurs (UBS, Credit Suisse) utilisent 3 colonnes. La détection binaire actuelle échoue.
3. **Extraction des sous-jacents** : le tableau des sous-jacents (actions/indices) n'est pas encore parsé en enregistrements structurés.
4. **Pas de normalisation des dates** : les dates sont retournées telles quelles (30.10.2029, 07/09/2027) sans conversion en ISO 8601.

12.2 Améliorations proposées

1. Implémenter la **continuation de lignes** : si la ligne $N + 1$ n'a pas de label (colonne gauche vide), son contenu est appendé à la valeur de la ligne N .
2. Ajouter un **score de confiance** par champ (high/medium/low) selon que la valeur vient d'une paire (high) ou d'un regex (medium).
3. **Validation croisée** : vérifier le checksum ISIN (Luhn), valider le code devise ISO 4217, vérifier que la maturité est dans le futur.
4. Tester sur des **émetteurs non-BIL** : Leonteq, BNP, UBS, Julius Baer.

13 Résumé des structures de données

```

1 @dataclass
2 class Word:
3     text: str
4     x0: float          # bord gauche
5     top: float         # bord superieur (Y)
6     x1: float          # bord droit
7     bottom: float      # bord inferieur
8     page: int
9
10 @dataclass
11 class Line:
12     words: list[Word]  # tries gauche-droite
13     y: float           # Y moyen de la ligne
14     page: int
15
16 @dataclass
17 class LabelValuePair:
18     label: str         # texte colonne gauche
19     value: str         # texte colonne droite
20     label_x: float     # position X du label
21     value_x: float     # position X de la valeur
22     y: float           # position Y
23     page: int
24
25 @dataclass
26 class ExtractionResult:
27     words: list[Word]
28     lines: list[Line]
29     pairs: list[LabelValuePair]
30     clean_text: str
31     page_count: int
32     tables_pdfplumber: list

```

Listing 40: Dataclasses principales

Conclusion

Cette pipeline démontre qu’une approche basée sur les coordonnées est supérieure à l’extraction de texte brut pour les documents semi-structurés. Le réglage clé — `x_tolerance=1` — résout le problème fondamental du collage de mots et permet une reconstruction fiable de la structure label-valeur.

La pipeline atteint 100% de précision sur les 3 documents de test BIL (EN, FR, DE) pour tous les champs présents dans les documents.

Nasser Kassioui
Mars 2026